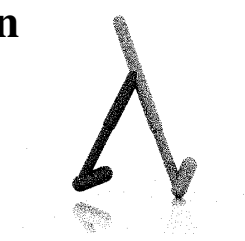


Aprendizaje por refuerzo profundo con aproximación de la función de valor de la acción Q

José Ridruejo Tuñón, Joaquín Mir Macías, Francisco Javier Ríos

Deep Reinforcement Learning



ABSTRACT

Este proyecto consiste en una implementación del algoritmo de DeepMind DQN para entornos de Deep Reinforcement Learning. Se ha desarrollado un código en Python utilizando las librerías de PyTorch, Gymnasium, MuJoCo y DM Control para entrenar los modelos en entornos como Walker2D, Hopper2d y Humanoid. Como input se ha utilizado la imagen RGB que proporciona el entorno para la visualización (también hemos añadido la opción de utilizar el vector de estados de las articulaciones), además se ha discretizado el espacio de acciones ya que DQN es discreto. La segunda parte de este proyecto consiste en la implementación de una serie de mejoras que agrupadas forman el algoritmo Rainbow DQN que busca arreglar las ineficiencias del algoritmo original. Finalmente se ha hecho un estudio de ablación donde se ha comparado el impacto de cada mejora respecto al algoritmo completo. Los entrenamientos han demostrado una clara ventaja de Rainbow sobre DQN base, consiguiendo un éxito con mucho menos tiempo de entrenamiento.

Key words: Deep Reinforcement Learning, Deep Q-Network, Rainbow DQN, MuJoCo, DeepMind Control Suite, Gymnasium, Visual Control, Continuous Control, Locomotion, Obstacle Avoidance

Repository: <https://github.com/pepert03/DQN-Rainbow-Pixel-Control>

1 INTRODUCCIÓN

Antes de explicar la estructura y metodología del proyecto, se va a hacer una breve definición del algoritmo de Deep Q-Networks con sus respectivas fórmulas, después se van a mencionar las 6 mejoras que conforman el algoritmo Rainbow DQN y que supuso una mejora en rendimiento y resultados sobre el original.

1.1 Deep Q-Networks (DQN)

El Q-learning tabular tradicional utiliza la ecuación de Bellman para actualizar el valor esperado de una acción a en un estado s :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

donde α es el *learning rate* y γ el factor de descuento.

Sin embargo, en entornos complejos donde mantener una tabla con todos los estados es inviable, DQN (Mnih et al., 2013) soluciona esto usando una Red Neuronal para aproximar la función Q , denotada como $Q_\theta(s, a)$. Se entrena minimizando el error temporal (TD error):

$$L(\theta) = \mathbb{E}_{s,a,r,s'} \left[\left(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \right)^2 \right]$$

Para evitar la inestabilidad de las redes neuronales en RL, DQN introdujo dos mecánicas clave:

- **Experience Replay:** Las transiciones se guardan en un buffer circular $\mathcal{D} = \{(s, a, r, s')\}$ y se muestrean en *batches* aleatorios. Esto rompe la correlación entre muestras consecutivas y reutiliza datos pasados.

- **Target Network:** Se utiliza una red neuronal separada (con pesos θ^-) para calcular el valor objetivo. Se actualiza copiando los pesos de la red principal cada ciertos *steps*, evitando que el objetivo a perseguir varíe constantemente en cada actualización de gradiente.

1.2 Rainbow DQN

Aunque DQN supuso un gran avance, tenía varios problemas: sobreestimación de valores, ineficiencia de muestreo y una exploración basada en ϵ -greedy. Rainbow DQN (Hessel et al., 2017) propuso unificar 6 mejoras independientes para solucionar estos cuellos de botella:

- **Double DQN:** Separa la selección de la acción de su evaluación para reducir el sesgo de sobreestimación. La red principal (online) elige la mejor acción, pero la *target network* es la encargada de evaluar su valor:

$$y = r + \gamma Q_{\theta^-}(s', \arg \max_{a'} Q_{\theta}(s', a'))$$

- **Prioritized Experience Replay (PER):** En lugar de muestrear aleatoriamente del buffer, da prioridad a las experiencias con mayor TD error, ya que representan resultados inesperados de los que el modelo puede aprender más rápido.

- **Dueling Architecture:** Divide el final de la red en dos flujos: uno estima el valor del estado $V(s)$ y el otro la ventaja de cada acción respecto a ese estado $A(s, a)$.

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')$$

Esto permite al modelo deducir qué estados son valiosos sin necesidad de aprender el efecto individual de cada acción en ellos.

- **N-step Returns:** Acumula las recompensas obtenidas tras n pasos en lugar de un solo paso. Esto propaga la señal de recompensa mucho más rápido hacia estados pasados, acelerando el aprendizaje.

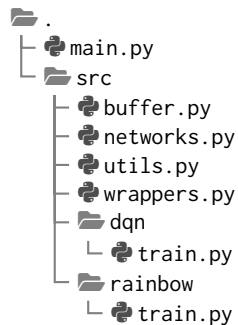
- **Distributional RL:** En lugar de predecir un único valor esperado, estima la distribución de probabilidad completa de los retornos. Esto captura la varianza y la incertidumbre estocástica del entorno.

- **Noisy Nets:** Elimina la exploración heurística (ϵ -greedy) inyectando ruido paramétrico directamente en los pesos de las capas lineales. La red aprende automáticamente a ignorar este ruido a medida que domina el entorno.

2 ESTRUCTURA DEL PROYECTO

Todos los modelos se han entrenado en Linux Ubuntu. Creemos que por diferencias en el renderizado entre sistemas operativos, los modelos tienen un rendimiento inferior cuando se evalúan en Windows.

La estructura del código ha sido la siguiente:



main.py: Solo se encarga de recibir los parámetros que se pasan en el comando de ejecución, que son el nombre del environment y la flag `'-train'` para entrenar.

buffer.py: Contiene los dos tipos de buffer que se utilizan en esta práctica: *Experience Replay* y *Prioritized Experience Replay* para Rainbow DQN.

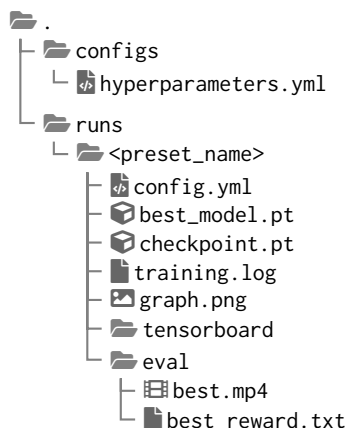
networks.py: Donde están los 3 módulos de PyTorch para el entrenamiento del modelo: el primero (DQN) es el MLP básico para inputs de estados; el siguiente (Pixel_DQN) es un MLP más una red convolucional para inputs de imágenes; y el último (NoisyLinear) se añade sobre los anteriores como una de las mejoras de Rainbow.

utils.py: Contiene funciones de carga de configuraciones y creación de gráficos.

wrappers.py: Agrupa todos los wrappers utilizados y las funciones de creación de environments. Inicialmente contenía wrappers

para rewards personalizados y creación de environments vectorizados para entrenar múltiples en paralelo. Sorprendentemente, la eliminación de estas implementaciones fue lo que consiguió el éxito de los modelos. `dqn/train.py` y `rainbow/train.py`: Contienen el bucle de entrenamiento principal del algoritmo. El primero incluye la clase `DQNAgent` y el segundo `RainbowAgent`, que hereda del anterior.

La estructura de la configuración y guardado de modelos se ha hecho sin utilizar librerías externas como Hydra (que, honestamente, habrían simplificado enormemente el código) y ha sido la siguiente:



La carpeta `runs` se genera automáticamente; basta con ejecutar un environment configurado en `hyperparameters.yml`. Un ejemplo de configuración puede ser así:

```
rainbow_pixel_walker2d:
  env_id: Walker2d-v5
  obs: pixel # options: pixel, state
  replay_memory_size: 200000
  mini_batch_size: 64
  epsilon_decay: 0.999995
  learning_rate: 0.0001
```

Para ejecutar un entrenamiento por primera vez, se deberá utilizar el siguiente comando:

```
uv sync
uv run python main.py <preset_name> --train
```

En este caso, el preset sería `rainbow_pixel_walker2d`. Se generará una carpeta con el mismo nombre en el directorio `runs/` que contendrá:

- Una copia de la configuración (que no recomendamos modificar para evitar errores).
- Un archivo `checkpoint.pt` que guarda todas las variables y pesos (excepto el *Experience Replay*) cada 100 épocas. Permitirá reanudar el entrenamiento si se llegase a parar (basta con volver a ejecutar el mismo comando), aunque el loop de entrenamiento es infinito.
- Un archivo `best_model.pt` que solo contiene la red neuronal y se guarda cuando se supera el *best score*.
- Un archivo de `training.log`.
- Un gráfico de media de rewards (`graph.png`) que se actualiza automáticamente durante el entrenamiento.
- Una carpeta de `tensorboard/` y una de `eval/` donde se guarda un vídeo de la mejor ejecución.

Para mostrar una visualización del `best_model`, hace falta ejecutar en modo evaluación:

```
uv run python main.py <preset_name>
```

3 APRENDIZAJE CON DQN

3.1 Metodología

Para la implementación, se siguió originalmente la serie de videos de Johnny Code *Implement Deep Q-Learning (DQN) in PyTorch* sobre DQN, aunque luego hubo que adaptar el código al entorno de Mujoco y implementar una CNN ya que el objetivo era procesar imágenes, no estados. Y aunque inicialmente se utilizó el código de los videos como guía, las multiples optimizaciones, reestructuración en módulos y las mejoras de rainbow, ha terminado cambiando completamente la estructura. Lo primero que se implementó fue el environment de cartpole con vector de estados ya que es el "MNIST" de este tipo de problemas de DRL y tarda solo un par de minutos en entrenar. La ejecución está guardada en el directorio runs/cartpole y el video de la mejor ejecución en runs/cartpole/eval/best.mp4 (esta misma estructura aplica a todas las siguientes ejecuciones).

Una vez completado la arquitectura base se procedió a añadir los cambios principales para conseguir ejecutarlo en Mujoco con observaciones RGB. Primero añadimos la CNN para el procesamiento de imágenes, y este fue el principal cuello de botella. Por lo que después de varias iteraciones sin buenos resultados, se redujo casi al mínimo numero de capas, se pasó a escala de grises para reducir dimensiones y se hizo un stack de 4 frames para darle al modelo sentido de dirección y de velocidad, y se dejó esta misma estructura para todos los environments:

```
Input: [B, 4, 84, 84] (4 grayscale frames)
→ Conv2d(4, 32, 8 × 8, stride=4) → ReLU
→ Conv2d(32, 64, 4 × 4, stride=2) → ReLU
→ Conv2d(64, 64, 3 × 3, stride=1) → ReLU
→ Flatten → Linear(3136, action_dim)
```

Otro cuello de botella fue que inicialmente se renderizaba a máxima calidad para luego hacer downscaling a las dimensiones adecuadas de 84×84 , esto permitía que a la hora de evaluar se pudiese ver a máxima calidad, sin embargo tener que renderizar tantos pixeles durante el entrenamiento era muy ineficiente. Finalmente se cambió la configuración del environment de Gymnasium para que renderizase nativamente en 84×84 mejorando los steps-per-second, aunque esto conllevó tener que hacer wrappers extra para la visualización en alta calidad y probablemente sea la causa de que los modelos tengan peor rendimiento al cambiar de OS en el que se entrenó.

Otro requisito necesario para la ejecución en Mujoco fue la discretización del espacio de acciones. Para ello usamos un wrapper que redujo cada articulación a 3 posibles movimientos (1, -1, 0) y puesto que solo mueve una articulación por step, se puede reducir a dos movimientos por articulación y un estado sin movimiento. Dejando 7 posibles acciones en el entorno de Hopper y 13 en Walker. Inicialmente se probó a permitir que el modelo pueda mover varias articulaciones por step, haciendo un vector de mayor tamaño que representase todas las combinaciones posibles dejando un vector de (posibles movimientos)^{nº de articulaciones}, después de varios días tratando de entrenar algún modelo sin llegar siquiera a hacer un paso se descartó esta idea.

3.2 Resultados y discusión

Para poder obtener buenos resultados, hubo que modificar el reward predeterminado de los environments. Como ya se ha mencionado antes, inicialmente se probó a hacer un reward desde cero recompensando el avance sin importar la velocidad, recompensando la estabilidad del walker2d calculando el ángulo del torso, simetría de la posición de las piernas... Sin embargo, todo esto dio malos resultados, además de ir en contra del objetivo de RL que es que aprendan a avanzar sin necesidad de dar instrucciones de como hacerlo. Al final la solución mas sencilla fue la más efectiva, solo se modificaron los pesos de los parámetros del reward original:

```
forward_reward_weight=0.5
ctrl_cost_weight=0.1
healthy_reward=1.25
```

Dando más importancia a mantenerse de pie que avanzar, ya que el modelo tendía a lanzarse al suelo. Otra modificación necesaria fue aumentar el numero de steps máximos por episodio ya que también hacía que el modelo tuviera 'prisa' por avanzar. Se aumentó a 20000 que aunque es más que suficiente y puede hacer que el modelo aprenda a mantenerse quieto, dio buenos resultados (aunque a veces encuentra un subóptimo quedándose quieto al inicio hasta que comienza a andar).

Los hiperparámetros elegidos fueron:

```
env_id: Hopper-v5
obs: pixel
discount_factor_g: 0.99
epsilon_decay: 0.999995
epsilon_init: 1.0
epsilon_min: 0.01
learning_rate: 0.0001
mini_batch_size: 64
network_sync_rate: 2000
replay_memory_size: 200000
```

Y prácticamente son los mismos en todas las configuraciones, ya que dieron buenos resultados en rainbow y con states. El epsilon decay se tuvo que poner cerca del 1 ya que en otro caso caía demasiado rápido. Otro hiperparámetro que dio problemas fue el replay_memory_size que si se aumentaba mucho daba errores de memoria de VRAM y aun así se tuvo que quitar del checkpoint ya que también causaba errores.

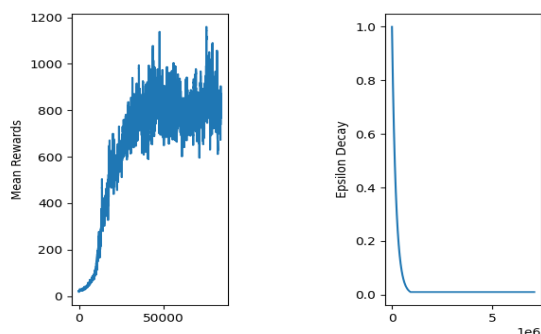


Figure 1. DQN Pixel Hopper: Reward/episode y epsilon/step

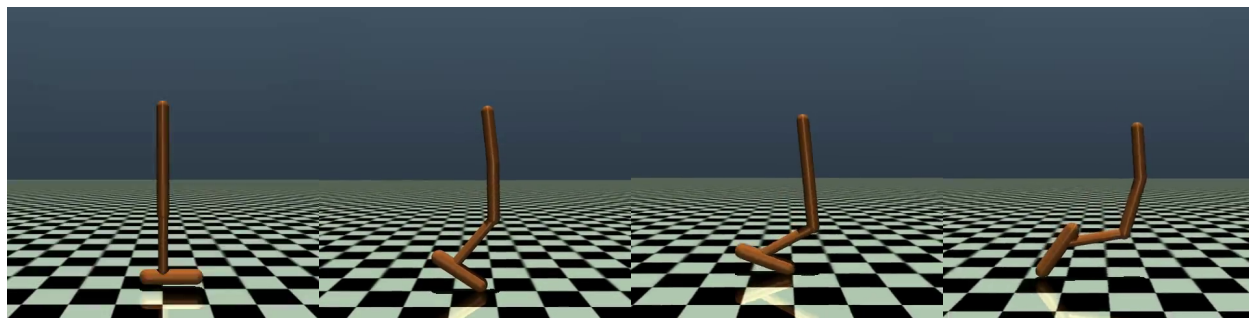


Figure 2. DQN Pixel Hopper: Mejor modelo (Best reward: 918)

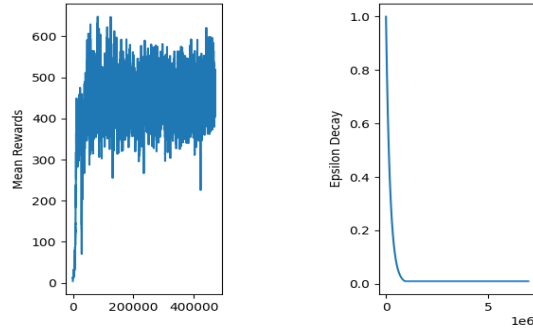


Figure 3. DQN Pixel Walker2d: Reward/episode y epsilon/step

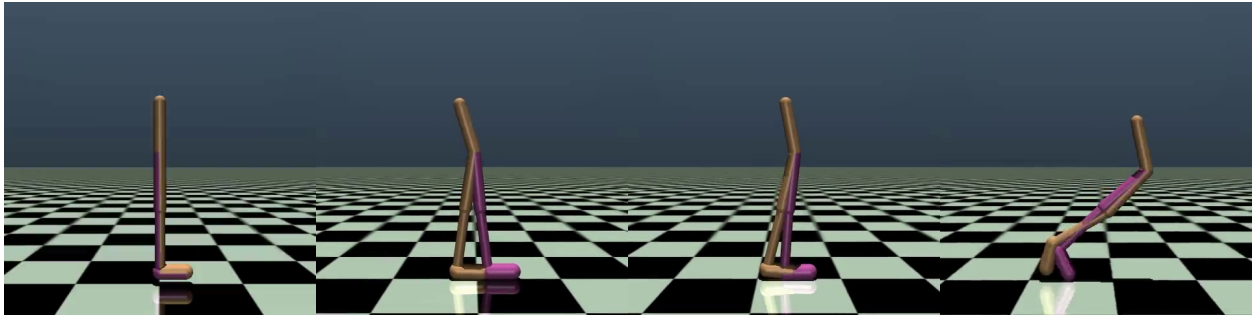


Figure 4. DQN Pixel Walker2d: Mejor modelo (Best reward: 1683)

3.3 Conclusiones

Los resultados muestran que el modelo aprende pero es extremadamente lento. La ejecución pixel_hopper estuvo 15 horas entrenando con un total de 7.000.000 steps y menos de 100.000 episodios y apenas llegaba al final en la mayoría de episodios. Claramente el modelo tenía varios cuellos de botella en el procesamiento de imágenes ya que entrenando con estados tardaba una fracción de lo que tarda con pixel y en apenas 1 hora el hopper era capaz de llegar al final. Estos fallos e ineficiencias se consiguieron arreglar con la implementación de Rainbow DQN.

4 APRENDIZAJE CON RAINBOW-DQN

4.1 Metodología

La serie de videos solo llegaba a implementar Double y Dueling DQN. Implementar n-steps, Prioritized Experience Replay y Noisy Nets fue relativamente fácil ya que era poco código o autocontenido en su propia clase.

En cambio, Distributional fue el más difícil ya que afectaba a la salida de las redes, y eso requería cambiar el comportamiento de otras mejoras como Dueling que también cambian la salida, además de tener que añadir la lógica de selección de átomos, proyección de Bellman, función de pérdida y optimización.

En cuanto a optimización, hubo que hacer varios cambios al algoritmo para conseguir acelerar el entrenamiento:

Optimización	Descripción
Train frequency = 4	Un paso de gradiente por cada 4 pasos del entorno; reduce el cómputo de entrenamiento mientras se recolectan datos más diversos.
Replay buffer en uint8	Las observaciones de píxeles se almacenan como uint8 en NumPy y se convierten a float32 solo al transferirse a la GPU para el entrenamiento.
Buffer circular	Uso de arrays de NumPy de tamaño fijo en lugar de listas de Python; elimina la sobrecarga de asignación de memoria por cada muestra.
Entrenamiento AMP	Uso de <i>torch.amp.autocast</i> + <i>GradScaler</i> en CUDA; <i>forward pass</i> se ejecuta en float16, manteniendo gradientes estables en float32.
Entornos vectorizados	Uso de <i>gymnasium.vector.SyncVectorEnv</i> para recolección de datos en paralelo; se descartó al no haber mejoras.

La mejora principal fue la de `train_frequency` que el paper original ya lo hacía, y aceleró $\times 4$ el tiempo de entrenamiento ya que la diferencia entre `steps` no aportaba información nueva para el entrenamiento, además, ya se hacía *frame stacking* de 4 frames por lo que era redundante no saltar 4 `steps`. El que peor resultado dio fue utilizar entornos vectorizados ya que aunque hacía muchos más `steps` por segundo, era mucho más ruidoso y no tenía un aprendizaje constante como con un solo entorno.

4.2 Resultados y discusión

Los hiperparámetros utilizados han sido los siguientes:

```
n_step: 3
num_atoms: 51
v_min: -200.0
v_max: 200.0
prioritized_replay_alpha: 0.6
prioritized_replay_beta: 0.4
prioritized_replay_beta_increment: 0.0
prioritized_replay_eps: 1e-6
sigma_init: 0.5
```

No ha hecho falta añadirlos en el yml ya que todos son los predeterminados y han funcionado sin problemas en los entornos de Hopper y Walker aunque para entornos más grandes como el Humanoid a lo mejor habría dado mejores resultados si se hubiera aumentado algún parámetro como el `v_max` para acciones con mucha recompensa.

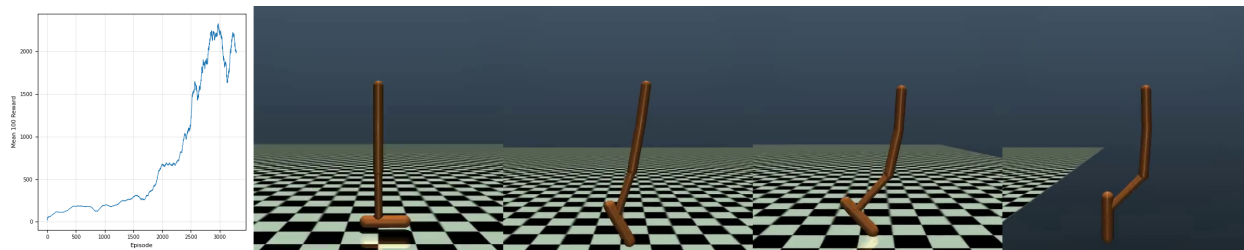


Figure 5. Rainbow DQN Pixel Hopper: Gráfica reward/episode y mejor modelo (Best reward: 3692)

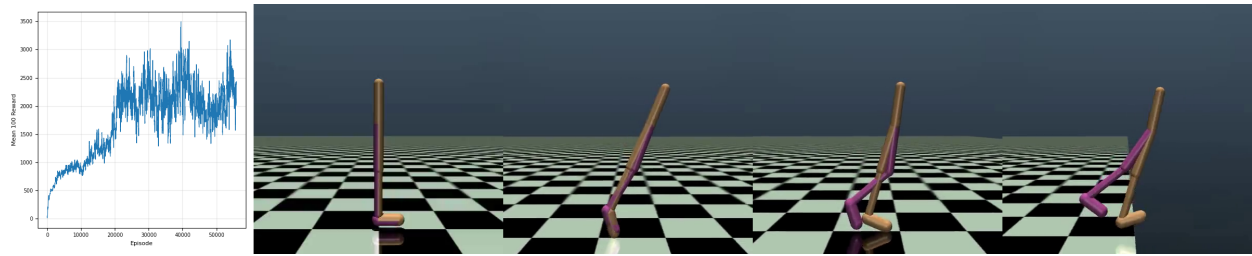


Figure 6. Rainbow DQN Pixel Walker2d: Gráfica reward/episode y mejor modelo (Best reward: 5847)

4.3 Conclusiones

Rainbow DQN finalmente dio muy buenos resultados consiguiendo llegar al final en los entornos de Walker2d y Hopper2d. Por hacer una comparación con los datos previos de Hopper en la DQN, con Rainbow en apenas 30 minutos ya conseguía avanzar hasta el final, con 300.000 *steps* y alrededor de 3.000 episodios.

5 ESTUDIO DE ABLACIÓN DE RAINBOW-DQN

5.1 Metodología

El estudio de ablación ha consistido en la eliminación de una de las mejoras de Rainbow y entrenamiento del environment Hopper durante un tiempo fijo (en la gráfica se muestra alrededor de una hora de entrenamiento, que es cuando Rainbow ha conseguido llegar al final). Se ha hecho el estudio con las 6 mejoras de Rainbow y se han comparado con el *baseline* y el Rainbow completo. Para cada entrenamiento solo ha hecho falta poner un *false* en alguna de las mejoras en la configuración:

```
no_double_pixel_hopper:
  # omitted
  enable_double_dqn: false
  enable_dueling_dqn: true
  enable_prioritized_replay: true
  enable_noisy_nets: true
  enable_distributional: true
  enable_n_step: true
```

5.2 Resultados y discusión

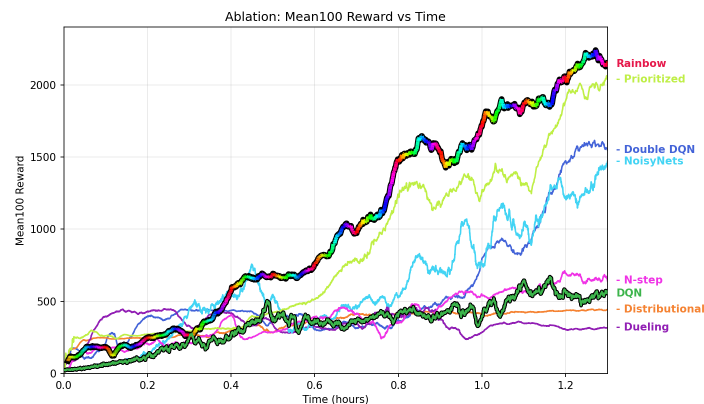


Figure 7. Ablation study on Hopper comparing Rainbow DQN against variants with one component removed.

Dado los resultados de la gráfica, se va a proceder a analizar los resultados y tratar de dar una explicación plausible.

Sin Dueling DQN: Este resultado ha sido inesperado ya que en el paper original Dueling es una de las mejoras que menos afecta al rendimiento, sin embargo, ha sido la ablación con peores resultados en nuestras pruebas. Puede ser que Dueling, al separar el valor de estado y la ventaja de la acción ayuda a estabilizar el entrenamiento y generaliza antes, y 1 hora de entrenamiento no es suficiente para que el resto de mejoras surjan efecto y superen el *baseline*. La segunda teoría es que debe haber un error en la lógica del código que hace que distributional y dueling tengan dependencias y eliminar una de las dos haga que el modelo falle, porque como ya se ha mencionado antes, distributional y dueling modifican la salida de las redes.

Sin Distributional RL: Este resultado se alinea más con los resultados del paper, parece que la eliminación de distributional puede tener un impacto en otras mejoras que se puedan beneficiar de saber la varianza, como podría ser las Noisy Nets. Aunque también puede ser un tema de dependencias con Dueling como ya se ha mencionado, porque las gráficas tienen un rendimiento parecido que hasta empeora el *baseline*.

Sin N-step: Esta es una de las mejoras principales y eliminarla tiene un claro impacto en el número de steps por segundo, lo cual inevitablemente afecta a las recompensas por episodio. Las actualizaciones de un solo paso tardan demasiado en propagar las recompensas a los estados previos, mientras que el aprendizaje n-step permite que esta señal fluya mucho más rápido.

Sin Prioritized Experience Replay: Otro resultado contradictorio respecto al paper ha sido PER. En la teoría, es una de las mejoras que más impacto tienen en el rendimiento, los resultados obtenidos dicen lo contrario. Es posible que el experimento no sea el más indicado para observar los beneficios de PER; al ser tan sencillo, es posible que el agente aprende antes de que el buffer se llene. También es posible que el beta estático haya capado la capacidad de corregir el Importance Sampling.

Sin Noisy Nets: El impacto de esta mejora no es más moderado en comparación con las anteriores ablaciones. Aunque a que la exploración basada en ruido paramétrico es superior a epsilon-greedy, en entornos pequeños el auto-ajuste de las Noisy Nets en exploración no es tan crítico.

Sin Double DQN: Curiosamente, según el paper, la eliminación de Double DQN es la que menos afecta al rendimiento, llegando en algunos casos a dar mejor resultado que el algoritmo completo. La justificación que dan es que el sesgo de sobreestimación del Q-learning ya está siendo reducido por el uso de Distributional RL, haciendo que la separación entre la selección y la evaluación de la acción pase a ser redundante.

5.3 Conclusiones

El estudio de ablación confirma que Rainbow no es simplemente un conjunto de mejoras sino que forman una red de pasos que se complementan y dependen entre sí. Y aunque es posible que tenga algunas redundancias, el algoritmo completo demuestra una ventaja enorme respecto al *baseline* y las mejoras independientes.

6 CONCLUSIONES

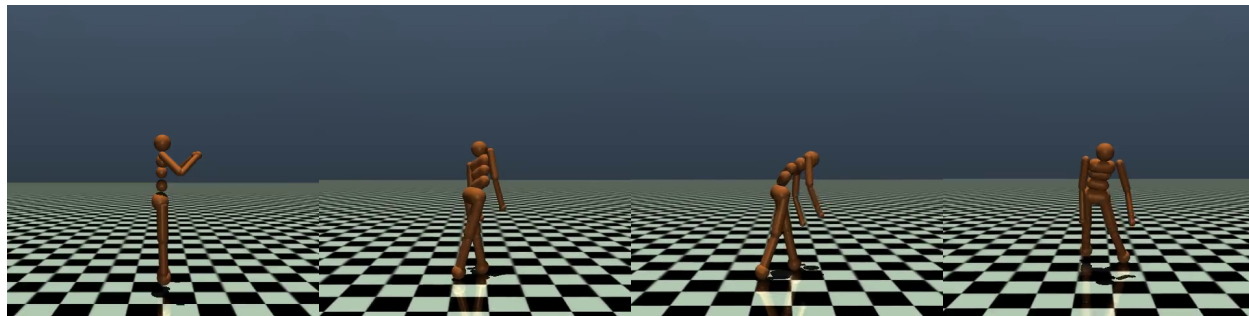


Figure 8. Rainbow DQN Pixel Humanoid: Mejor modelo (Best reward: 329)

Se han implementado los algoritmos de DQN y Rainbow DQN en Pytorch siguiendo los papers originales, y se ha aplicado a entornos de locomoción de Mujoco con la librería Gymnasium. Se ha conseguido entrenar exitosamente los entornos de Walker y Hopper, demostrando la ventaja de Rainbow sobre DQN. Se han analizado el impacto de las mejoras independientes que componen Rainbow. Como futuras mejoras quedaría conseguir entrenar exitosamente Humanoid: se trató de entrenar haciendo un wrapper de Frozen Joints para reducir la complejidad, pero solo se consiguió que diera un par de pasos. Además habría que probar estos mismos agentes en entornos con obstáculos para conseguir un resultado más robusto.

REFERENCES

- Code, J., 2024, Deep reinforcement learning in python: <https://www.youtube.com/playlist?list=PL58zEckBH8fCMIVzQCRSZVPUp3ZAVagWi>. (YouTube playlist).
- Hessel, M., J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. G. Azar, and D. Silver, 2017, Rainbow: Combining improvements in deep reinforcement learning: arXiv preprint arXiv:1710.02298.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, 2013, Playing atari with deep reinforcement learning: arXiv preprint arXiv:1312.5602.